

Supervised Learning Algorithms

Kauã de Oliveira Ribeiro
Instituto Nacional de Telecomunicações (INATEL)
Santa Rita do Sapucaí, Brasil
kaua.ribeiro@ges.inatel.br

1. Introduction

Machine Learning is the science and art of programming computers giving them the ability to learn from data [1]. The field is broadly divided into 3 paradigms:

- **Supervised:** A labeled dataset is available (input x , output y). The goal is to construct a function $y = f(x)$ to predict the label y of a new sample x (not previously observed) from the same distribution.
- **Unsupervised:** The dataset is not labeled. The goal is to discover properties of the dataset's structure (characterized by the density $p(x) = p(x_1, \dots, x_n)$).
- **Reinforcement:** Unlike the previous paradigms, here there is no type of training data, whether labeled or not. Learning is done by trial and error and must balance discovery (exploration) and exploitation. The goal is to discover and execute the best actions in each situation in order to maximize the reward obtained.

This document explores the fundamental algorithms of supervised learning, highlighting how they work, their key properties, and practical applications, while also presenting the metrics required for robust model evaluation.

1.1. Organization

This paper is organized as follows. Section I presents the introduction and the fundamental paradigms of machine learning. Section II discusses supervised learning algorithms, detailing both regression and classification methods, their decision boundaries, and limitations. Section III explores the evaluation of machine learning models, presenting key performance metrics for both regression and classification tasks. Finally, Section IV concludes the paper, summarizing the main insights regarding algorithm selection.

2. Supervised Learning

As stated in the introduction, supervised machine learning algorithms are designed to learn from labeled training data. The model receives a set of input features (x) and their corresponding expected outputs (y). The objective is to learn

a mapping function $f(x)$ capable of accurately predicting the output $\hat{y}$ for new, unseen data. These tasks are fundamentally categorized into two main domains: regression and classification.

2.1. Train/Test Split

To objectively evaluate how well a model has learned, the dataset cannot be used entirely for training. Instead, it is standard practice to divide the data into two mutually exclusive subsets: a **training set** and a **testing set**. The algorithm uses the training set to find patterns and adjust its internal parameters. Once trained, the model is exposed to the testing set—data it has never seen before—to evaluate its true predictive capability.

2.2. Underfitting and Overfitting

The ultimate goal of any machine learning model is generalization. However, two primary challenges can prevent this [1]:

- **Underfitting:** Occurs when a model is too simple to capture the underlying mathematical structure of the data. It performs poorly on both the training set and the testing set because it fails to learn the essential correlations.
- **Overfitting:** Occurs when a model is excessively complex. Instead of learning the general patterns, it memorizes the training data, including its specific noise and anomalies. Consequently, it achieves near-perfect accuracy on the training set but fails dramatically when applied to the testing set.

The exact methods and metrics used to quantify these errors and evaluate model performance depend heavily on the specific task and the model structure. The following subsections detail the primary algorithms used in regression and classification, while Section III provides an in-depth analysis of the evaluation metrics used to quantify their success.

2.3. Regression

Regression algorithms are mathematical models designed to predict continuous numerical values for a target

variable y based on a set of input features x . These models focus on learning a continuous mapping function to approximate the underlying data distribution.

2.3.1. Linear and Multiple Linear Regression. Linear regression is one of the simplest possible algorithms. It aims to construct a straight line that best fits the data by minimizing the residual errors between the predicted and actual values. In its simplest form, Simple Linear Regression models the relationship between a single input x and the output y using the equation $y = wx + b$, where w represents the weight of the input and b is the bias factor that shifts the curve to the correct position.

When the problem involves several variables, the model evolves into a Multiple Linear Regression algorithm. It maintains the first-degree linearity but incorporates multiple inputs, expressed as $y = b + w_1x_1 + w_2x_2 + \dots + w_nx_n$, where w represents the different weights for each input variable x .

In their analytical form, which is based on Ordinary Least Squares, these models do not require hyperparameters. However, when trained using optimization algorithms like Gradient Descent, the *Learning Rate* determines the step size for weight updates [1]. Additionally, if trained with regularization techniques like Ridge or Lasso, the *Regularization Penalty* (α or λ) becomes a crucial hyperparameter that shrinks weights to prevent the model from relying too heavily on a single feature.

The regression curves of the resulting models are strictly **linear**. For simple regression, it produces a straight line, while for multiple regression, it creates a flat, multi-dimensional hyperplane.

These models are highly advantageous due to their fast training times, low computational cost, and excellent interpretability, as the weights directly indicate feature importance. However, they are severely limited when dealing with non-linear correlations, often resulting in underfitting. Furthermore, they are highly sensitive to outliers, as extreme values can drastically skew the slope of the fitted line.

2.3.2. Polynomial Regression. Polynomial Regression is an extension of linear regression engineered to discover non-linear relationships between the inputs and the output. It works by mathematically transforming the original input feature x into higher-degree polynomial terms, such as $[1, x, x^2, x^3, \dots, x^n]$. After this transformation, a standard linear regression algorithm is applied to the new feature set. Consequently, even though the trained model remains linear with respect to its parameters, it acts non-linearly with respect to the original input x .

The defining hyperparameter of these algorithms is the *Polynomial Degree* (n). It directly defines the predicted curve of these models; a degree of 2 forms a parabolic curve, while higher degrees create increasingly complex, wavy curves capable of intersecting more data points.

They produce **non-linear and smooth** curves. Because they rely on polynomial mathematical functions, the transi-

tions along the curve are continuous and gradually trace the dispersion of the data.

They are extremely useful for solving regression problems that are not linear and require exponential or curvilinear growth, where a straight line would inevitably fail. However, one of their biggest limitations is their extreme vulnerability to overfitting. If the polynomial degree is set too high, the model will memorize the training data, including its noise. This results in a highly irregular curve that performs poorly on unseen data and diverges wildly at the edges of the dataset [1]. Conversely, if the degree is too low, the model will suffer from underfitting, remaining too simple to capture the real underlying patterns of the data.

2.4. Classification

While regression models predict continuous values, classification algorithms are designed to categorize input data into a finite number of discrete classes. The primary objective of a classifier is to learn a mapping function that establishes a decision boundary. This mathematical frontier divides the feature space into distinct regions, with each region corresponding to a specific class label.

Interestingly, the mechanics behind classification often rely on continuous mathematical functions. For instance, some algorithms calculate a continuous probability score and then apply a threshold to assign the final discrete category. Depending on the chosen algorithm and the complexity of the dataset, these decision boundaries can range from simple straight lines to highly complex, non-linear shapes.

2.4.1. Binary Classification. Binary Classification is the fundamental form of classification algorithms, where the model must choose between exactly two possible classes.

One of the most prominent binary classification algorithms is **Logistic Regression**. Despite its name, it is a classification model that calculates a linear combination of the input features and applies the Sigmoid function ($\sigma(z) = \frac{1}{1+e^{-z}}$) to output a probability between 0 and 1 [1]. If this probability exceeds a certain threshold, the instance is assigned to the positive class.

The classification threshold, typically set at 0.5, acts as the primary hyperparameter, allowing engineers to tune the model's sensitivity. Additionally, if the model is trained with regularization techniques, the *Regularization Penalty* (C) can be adjusted to prevent overfitting by penalizing excessively large weights.

The resulting model produces a strictly **linear and smooth** decision boundary. The geometrical boundary dividing the classes is a straight line, or a hyperplane in higher dimensions, but the transition between the two regions is probabilistic and smooth, rather than a hard cutoff.

This model is highly advantageous when the probability or confidence of the prediction is required, such as in medical risk assessments. However, it fails drastically on non-linearly separable datasets, as the underlying mathematical equation cannot curve its decision boundary.

2.4.2. Multiclass Classification. Unlike binary problems, multiclass classification involves categorizing data instances into three or more distinct classes. To achieve this, several algorithmic approaches can be employed, varying from multiple linear boundaries to complex spatial distance methods.

The Perceptron is one of the foundational algorithms in this domain. Operating as a linear classifier, it assigns mathematical weights to input features, computes their sum, and passes the result through a step function to establish a classification. While originally designed for binary tasks, its logic is frequently extended to handle multiclass problems through strategies such as One-vs-Rest (OvR).

Its fundamental hyperparameters are the *Learning Rate* (η) and the number of *Epochs*. The learning rate defines the step size for weight updates during training; a value that is too high causes instability, while a value that is too low significantly slows down convergence.

In a multiclass scenario, the Perceptron generates **linear and abrupt** decision boundaries. It essentially draws multiple rigid straight lines to separate the different classes. Because it relies on a step function, there is no smooth probability transition, meaning the classification changes abruptly as soon as the boundary is crossed.

The algorithm is computationally fast and effective for perfectly linearly separable data. However, it struggles significantly, often failing to converge entirely, if the classes overlap or require non-linear separation.

In contrast, **K-Nearest Neighbors (KNN)** represents a fundamentally different, non-parametric approach. KNN is an instance-based learning algorithm that does not compute weights or construct a generalized mathematical equation during training. Instead, it classifies a new, unseen data point based strictly on its spatial proximity to the K closest examples in the training dataset, assigning the most frequent class among those neighbors.

The most critical hyperparameter in this algorithm is K , which represents the number of neighbors. A small K creates a model that is highly sensitive to noise in the data, leading to severe overfitting. Conversely, a larger K smooths the decision area and provides a more stable classification. The choice of the spatial metric, commonly the Euclidean distance, also significantly impacts the results.

KNN produces a **highly non-linear and fragmented** decision boundary. Because classification relies entirely on localized spatial proximity, the boundary perfectly wraps around clusters of data, forming irregular, organic shapes rather than straight lines.

This method is excellent for smaller datasets with complex, non-linear relationships, as it makes no assumptions about the data's distribution. Its major limitation is inference time as computing the distance to every training sample is computationally expensive and memory-intensive for large or high-dimensional datasets.

3. Evaluating Machine Learning Models

Evaluating a machine learning model is as critical as training it. A model that performs flawlessly on its training

data may simply be memorizing it (overfitting) rather than learning generalized patterns [1]. To prevent this illusion of performance, techniques such as **K-Fold Cross-Validation** are employed. This method randomly splits the training set into K distinct subsets (folds), training the model K times, each time evaluating it on a different fold while training on the remaining ones. This ensures a robust, unbiased estimation of the model's performance on unseen data.

Once an appropriate validation strategy is established, the choice of evaluation metrics depends fundamentally on whether the model is performing regression or classification.

3.1. Regression Metrics

To evaluate a regression model, we measure the distance between the predicted values ($\hat{y}$) and the actual values (y). The most common metric is the **Mean Squared Error (MSE)**, which penalizes larger errors by squaring them:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

A widely used derivative is the **Root Mean Squared Error (RMSE)**. By taking the square root of the MSE, the error metric is returned to the original unit of the target variable, making it significantly easier to interpret in practical scenarios.

3.2. Classification Metrics

Evaluating a classifier is often significantly trickier than evaluating a regressor, as accuracy alone can be misleading, especially in imbalanced datasets. The foundation of classification evaluation is the **Confusion Matrix**, which divides predictions into four categories: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).

From this matrix, we derive critical metrics:

- **Precision:** Measures the accuracy of positive predictions. It is crucial when the cost of a False Positive is high.

$$Precision = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity):** Measures the ratio of actual positives that were correctly identified. It is vital when missing a positive instance (False Negative) is dangerous, such as in medical diagnoses.

$$Recall = \frac{TP}{TP + FN}$$

- **F1 Score:** The harmonic mean of Precision and Recall. It provides a single metric that balances both concerns, which is especially useful when comparing multiple models.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

3.3. The Precision/Recall Trade-off

In an ideal scenario, a model would have both perfect precision and perfect recall. In reality, increasing precision typically reduces recall, and vice versa [1]. Classifiers compute a score based on a decision function; if that score is greater than a predefined threshold, it assigns the instance to the positive class. Adjusting this decision threshold allows engineers to tune the model according to the specific business or scientific requirements of the project.

3.4. ROC Curve vs. PR Curve

The Receiver Operating Characteristic (ROC) (Receiver Operating Characteristic) curve plots the True Positive Rate (Recall) against the False Positive Rate (FPR) across different classification thresholds. The Area Under the Curve (Area Under Curve (AUC)) provides an aggregate measure of performance, where an AUC of 1.0 represents a perfect model, and 0.5 indicates random guessing.

However, a critical evaluation rule is knowing when to use the ROC curve versus the Precision/Recall (PR) curve. As a rule of thumb, the PR curve should be used whenever the positive class is rare, or when the cost of False Positives is more significant than False Negatives [1]. In heavily imbalanced datasets, an ROC curve might look excessively optimistic because the abundance of True Negatives skews the FPR, masking the model's actual struggle to correctly identify the rare positive instances.

4. Conclusion

As seen in the last sections, the development of effective machine learning solutions requires a fundamental understanding of the available learning paradigms. Supervised learning is the mandatory approach when a labeled dataset is available to map inputs to specific targets. Conversely, when data is unlabeled, unsupervised learning must be used to discover hidden structures and clusters. For scenarios involving sequential decision-making in dynamic environments, reinforcement learning is applied, relying on trial, error, and reward maximization rather than static datasets.

Within the supervised learning paradigm, selecting the appropriate algorithm depends entirely on the nature of the target variable and the geometrical complexity of the data. For continuous numerical predictions, regression models are the standard. Simple and Multiple Linear Regression provide highly efficient solutions for linear relationships, whereas Polynomial Regression offers the mathematical flexibility to trace curvilinear trends, though it requires strict tuning of its degree to prevent severe overfitting.

For discrete categorization tasks, the choice of algorithm depends on both the decision boundary shape and the number of target classes. For binary classification problems, Logistic Regression is highly effective, constructing a smooth, linear boundary that assigns probabilistic outcomes between two distinct categories. However, when the problem expands to multiclass classification, the approach must adapt

to the data's geometry. If the multiple classes are linearly separable, the Perceptron can effectively divide the space using multiple rigid, abrupt linear boundaries.

Conversely, for complex, non-linear multiclass problems, rigid straight lines are insufficient. In these scenarios, K-Nearest Neighbors (KNN) can be employed. By utilizing spatial distance metrics, KNN forms fragmented, non-linear boundaries that organically wrap around data clusters, demanding careful tuning of the K parameter to balance noise sensitivity. Ultimately, successful model deployment relies on matching the mathematical mechanics and boundary behaviors of these algorithms to the specific characteristics of the problem at hand.

References

- [1] A. Géron, *Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly Media, 1st ed., 2017.